

Exam Model Answers — copy-paste ready

One section per expected exam question. Under each question: a first-person **Copy-paste answer** (drop it into the report template, then personalize), **Why & how — justification add-ons** (standalone sentences in the decision → rejected alternative → concrete benefit pattern the lecturer wants, What-To-Expect p.4), **Adapt to your lab** (exactly what to swap in, with [bracketed placeholders] — the placeholders are intentional), and **Key terms to drop** (textbook vocabulary with one-line definitions). The first five questions are the verbatim sample questions from the lecturer (What-To-Expect p.3–4, p.6); the rest are derived from the seven emphasized topic areas. Theory is grounded in the course decks and labs and cited as (Deck p.N); keep or strip the citations as you prefer. Anything beyond slide content is flagged "(beyond slides — practical knowledge)".

Three rules for using this file at the exam. First, **never paste a claim your repository cannot back**: the repository is graded alongside the report (What-To-Expect p.5), so swap every generic claim for your real branch, class, and commit names before moving on — the "Adapt to your lab" bullets list exactly which parts are swappable. Second, **answer the question asked, not the question prepared**: if the template's wording differs from a heading here, search Lookup for the template's exact wording and let the closest section serve as raw material rather than a final answer. Third, **every paragraph you submit should contain a why**: if a pasted answer reads as pure description after personalization, append one add-on sentence from the matching "Why & how" block. The exam-day workflow, allowed-aids rules, and the AI ban are covered in the companion guide `90-exam-what-to-expect.md`.

What is meant by a refactoring pattern?

Copy-paste answer

A refactoring pattern is a named, catalogued, behaviour-preserving code transformation: a reusable recipe that says *when* a given structural problem occurs, *what* sequence of edits fixes it, and *why* the result is better. Refactoring itself is defined as "a change made to the internal structure of software to make it easier to understand, and cheaper to modify" where "the observable behavior of the software should not be changed" (Refactoring1 p.3). A refactoring *pattern* packages one such change under a name — Extract Method, Move Method, Extract Class, Replace Conditional with Polymorphism — so that developers share a vocabulary and can apply proven mechanics instead of improvising (Refactoring1 p.9–58). Fowler's catalog groups about fifty of these into seven categories, from Composing Methods to Big Refactorings (Refactoring1 p.9). The pattern idea follows Alexander's definition: a three-part rule connecting a context, a problem, and a solution (HighLevelRefactoring p.7). Each catalog entry has exactly that shape: the context is the smell that triggers it (a Long Method, a Switch Statement), the problem is why that structure resists change, and the solution is the mechanical transformation. Kerievsky extends this with composite, high-level refactorings — sequences of low-level refactorings that introduce an entire design pattern, solving design problems gradually with an overall plan (HighLevelRefactoring p.16). In my own project I applied refactoring patterns from this catalog rather than ad-hoc rewrites, because each small named step is less likely to go wrong and keeps the system fully working between steps (RefactLab p.1).

Why & how — justification add-ons

- I used named catalog refactorings instead of free-form rewriting because each named move has known mechanics and a known risk profile, so behaviour preservation can be verified step by step rather than

hoped for at the end (Refactoring1 p.3; RefactLab p.1).

- I performed a sequence of small refactorings instead of one big-bang restructuring because "since each refactoring is small, it's less likely to go wrong" and the system stays fully working after every step (RefactLab p.1).
- I preferred tool-automated refactorings in the IDE over manual editing — Kerievsky's "Automation First" heuristic — because the IDE preserves behaviour mechanically while manual edits depend on my attention (HighLevelRefactoring p.20).
- I treated the pattern's structure diagram as an example rather than a specification, because Vlissides notes a pattern's diagram "is just an example, not a specification", which let me adapt the move to my actual class shapes (HighLevelRefactoring p.18–19).
- I checked the refactoring against its triggering smell first, instead of applying patterns speculatively, because over-applied patterns are themselves a smell (Speculative Generality) and Kerievsky's catalog includes moves *away* from patterns for exactly that reason (Refactoring1 p.7; HighLevelRefactoring p.21–22).
- I documented each applied pattern by its catalog name instead of describing the edit informally, because the shared vocabulary lets a reviewer verify the mechanics against the catalog without reading my whole diff (Refactoring1 p.11–58).

Adapt to your lab

- Name [the refactoring pattern you actually applied, e.g. Extract Method / Replace Conditional with Polymorphism] and [the class in your JHotDraw fork where you applied it].
- State [the smell that triggered it, e.g. Long Method / Duplicated Code] so the pattern's context–problem–solution shape is visible in your answer.
- Point at [the commit or feature branch] where the refactoring can be inspected.
- If you used a Kerievsky composite refactoring toward a pattern, name [the design pattern you refactored toward, e.g. Strategy or Template Method].
- If the exam asks for the definition only, the first three sentences of the copy-paste answer stand alone as a complete theory answer.

Key terms to drop

- **Refactoring** — internal-structure change that keeps observable behaviour identical (Refactoring1 p.3).
- **Behaviour preservation** — the hard constraint: same inputs, same outputs and side effects; what makes refactoring safe and verifiable (Refactoring1 p.3).
- **Catalog refactoring** — a named entry in Fowler's catalog with mechanics and motivation, e.g. Extract Method (110) (Refactoring1 p.11).
- **The seven categories** — Composing Methods; Moving Features; Organizing Data; Simplifying Conditionals; Making Method Calls Simpler; Generalization; Big Refactorings (Refactoring1 p.9).
- **Pattern (Alexander)** — "a three-part rule" relating context, problem, and solution; both a process and a thing (HighLevelRefactoring p.7).
- **Composite / high-level refactoring** — a planned sequence of low-level refactorings that introduces a design pattern (HighLevelRefactoring p.16).
- **Algebra of refactoring** — Kerievsky's framing: patterns are the word problems, refactorings the algebra you solve them with (HighLevelRefactoring p.8–10).
- **Rule of Three** — refactor the third time you meet the same duplication; don't abstract on the first occurrence (Refactoring1 p.5).

How do you identify a code smell?

Copy-paste answer

A code smell is a surface symptom in source code that usually indicates a deeper design problem and points to a candidate refactoring — Fowler calls them "symptoms of bad code" (Refactoring1 p.6). A smell is not a bug: the program can run perfectly and still smell. I identify smells in three complementary ways. First, by recognising the named catalog symptoms while reading: Fowler lists 22, including Duplicated Code, Long Method, Large Class, Long Parameter List, Switch Statements, Feature Envy, and Comments used to compensate for unclear code (Refactoring1 p.6–8). Second, by measuring against concrete thresholds, because numbers catch what reading misses: units longer than 15 lines of code, more than 4 branch points per unit, duplicated blocks of 6 lines or more, more than 4 parameters (BetterCode p.10, p.16, p.23, p.25). Third, with tooling: in the refactoring lab I installed SonarLint in the IDE and let static analysis flag smells in my JHotDraw code relative to my change request (RefactLab p.1). I used tool findings as triggers, not verdicts — I confirmed each one by asking *why* the structure resists change (does this duplication mean a fix must be made twice? does this switch mean every new case edits old code?) before refactoring, because a smell only matters when it stands in the way of an actual change. Naming the smell then tells me the fix: Long Method points to Extract Method, Switch Statements to Replace Conditional with Polymorphism (Refactoring1 p.6–8).

Why & how — justification add-ons

- I combined human reading with SonarLint instead of relying on the tool alone, because static analysis flags symptoms but cannot judge whether the structure actually obstructs my change — that judgement needs the change request as context (RefactLab p.1).
- I used the SIG thresholds (15 LOC, 4 branch points, 6-line duplication, 4 parameters) instead of gut feeling, because measurable limits make the smell diagnosis repeatable and defensible (BetterCode p.10, p.16, p.23, p.25).
- I prioritised smells in code I actually had to change rather than cleaning everywhere, because a smell's cost is paid only when the code is read or modified — complexity that is never touched accrues no interest (BeyondTechnicalDebt p.12).
- I distinguished Divergent Change (one class changes for many reasons) from Shotgun Surgery (one change touches many classes), because they are mirror-image smells with opposite fixes — split the class versus gather the scattered behaviour (Refactoring1 p.6).
- I treated explanatory comments as a smell rather than documentation, because a comment compensating for bad code is a deodorant — the fix is Extract Method plus a self-documenting name, after which the comment becomes unnecessary (Refactoring1 p.8).
- I re-ran the smell check after each fix instead of assuming the fix was clean, because refactorings can introduce new smells of their own — the catalog's inverse pairs exist precisely because both directions of a structure can smell (Refactoring1 p.6; HighLevelRefactoring p.21–22).

Adapt to your lab

- Name [the smell you found in your JHotDraw code, e.g. Long Method in a tool class / Duplicated Code across figure classes] and [the class/method it lived in].
- State [how you found it: SonarLint warning / reading during concept location / a metric threshold it broke].
- Name [the refactoring you applied to remove it] and [the branch/commit holding the fix].

- If your lab used Kerievsky's smell vocabulary, mention that the smell descriptions came from [Ker05] Chapter 4 (RefactLab p.1).

Key terms to drop

- **Code smell** — a surface indication that usually corresponds to a deeper design problem; a trigger for refactoring, not a defect (Refactoring1 p.6).
- **The 22 smells** — Fowler's full symptom list, from Duplicated Code to Comments (Refactoring1 p.6–8).
- **Smell** → **refactoring mapping** — each smell names its cure: Feature Envy → Move Method; Data Clumps → Introduce Parameter Object; Switch Statements → Replace Conditional with Polymorphism (Refactoring1 p.6–8).
- **Cyclomatic complexity** — branch points + 1; the measure behind the " ≤ 4 branch points per unit" guideline (BetterCode p.16).
- **Duplicated block** — ≥ 6 identical lines occurring more than once; the hard floor of the duplication detector (BetterCode p.23).
- **SonarLint** — the IDE static-analysis plugin the lab uses to find smells in JHotDraw (RefactLab p.1).
- **Kerievsky smells** — five additions to Fowler's list, e.g. Conditional Complexity and Solution Sprawl (HighLevelRefactoring p.12–13).
- **Hotspot** — complicated code you must work with often: high complexity $\times$ high change frequency, the smell worth fixing first (BeyondTechnicalDebt p.12).

How did you create your pipeline?

Copy-paste answer

I created my continuous-integration pipeline with GitHub Actions, following the course CI lab (CILab p.1). I added a workflow file — a `*.yaml` — to the repository path `.github/workflows/`, which is where GitHub Actions looks for instructions telling the CI service what to do (CILab p.1). In the workflow I configured the project to build automatically **for each pull request**, using Maven as the build tool, and to **execute the tests automatically** as part of every build (CILab p.1). To resolve shared jar dependencies from the GitHub Packages registry, I created a `.maven-settings.xml` file in the project root, following the "Working with the Apache Maven registry" guide (CILab p.1). I chose to trigger on pull requests instead of building manually before hand-ins because building every change is the CI principle "commit frequently and build every commit" — frequent automated builds shrink the time between introducing a defect and discovering it (ContinuousIntegration p.3, p.7). I made the build self-testing — the build runs its own unit tests — because a green build should mean *working* software, not merely compiling software, and unit tests are the best method for verifying builds: they are fast (no database or file system) and focused enough to pinpoint problems (ContinuousIntegration p.9). The result is that the repository, not any developer's machine, is the source of record, and the build server is the final authority on whether the code is stable (ContinuousIntegration p.6).

Why & how — justification add-ons

- I used Maven rather than building from the IDE because Maven makes the build reproducible from a clean checkout on any machine, which is the precondition for a build server settling "it works on my box" disputes (ContinuousIntegration p.6; CILab p.1).
- I triggered the workflow on pull requests instead of on a nightly schedule because "if it hurts, do it more often": difficult activities become routine when done frequently, and per-change builds attribute each failure to one small diff (ContinuousIntegration p.7).

- I put the tests inside the pipeline instead of running them only locally because individual programmers are less than 50% efficient at finding their own bugs, so an independent automated check catches what I miss (ContinuousIntegration p.8).
- I kept the pipeline's checks to fast unit tests rather than end-to-end system tests because system tests take minutes to hours, and a slow build defeats the point of building every commit — "keep the build fast" (ContinuousIntegration p.9, p.11).
- I stored the workflow file in the repository itself rather than configuring the server by hand, because versioning the pipeline alongside the code means the build instructions evolve with the code and are themselves reviewable (beyond slides — practical knowledge).
- I started from the lab's reference guide ("Building and testing Java with Maven") instead of writing the workflow from scratch, because a documented baseline configuration reduces setup errors and let me spend my time on the project-specific parts — the test execution and the registry settings (CILab p.1).

Adapt to your lab

- Name [your workflow file, e.g. `maven.yml`] and quote [its trigger block: `pull_request` / `push` to which branches].
- List [the steps your workflow actually runs: checkout, set up JDK, `mvn` goals such as `clean install` or `verify`].
- State whether you wired [GitHub Packages via `.maven-settings.xml`] and why (shared jars), or note that your project resolved everything from Maven Central.
- Mention [one real pipeline run: a PR where the build caught a failure, with the branch name] as evidence the pipeline did its job.
- State [the JDK and Maven versions your workflow sets up] to show the build environment is pinned, not accidental.

Key terms to drop

- **Continuous integration** — merging all working copies to a shared mainline several times a day, each integration verified by an automated build (ContinuousIntegration p.2; CILab p.1).
- **GitHub Actions workflow** — the `*.yml` under `.github/workflows/` that tells the CI service what to do (CILab p.1).
- **Self-testing build** — a build that runs its own tests, so green means working, not just compiling (ContinuousIntegration p.9).
- **Build server** — the final authority on stability and quality; it only gets code from the repository (ContinuousIntegration p.6).
- **Maven** — the Java build tool used to compile, test, and install the project reproducibly (CILab p.1; IntroLab p.1).
- **GitHub Packages** / `.maven-settings.xml` — the shared jar registry and the settings file that wires Maven to it (CILab p.1).
- **CI history** — Grady Booch coined the term in 1991; Extreme Programming advocated integrating tens of times per day (CILab p.1).
- **"If it hurts, do it more often"** — the agile principle justifying building every commit (ContinuousIntegration p.7).

How does the pipeline work?

Copy-paste answer

My pipeline follows the standard CI cycle: commit → build → test → report, looping back into development (ContinuousIntegration p.2). Concretely: when I push a branch and open a pull request, GitHub Actions detects the event and reads my workflow file in `.github/workflows/` (CILab p.1). The runner checks out the repository — the source-code repository is the source of record, and the build server only ever gets code from it (ContinuousIntegration p.6) — sets up the Java environment, and runs the Maven build. Maven compiles the project from a clean state, resolves dependencies (in my setup, shared jars come from the GitHub Packages registry via `.maven-settings.xml`, CILab p.1), and then executes the automated test suite, because the build is configured to be self-testing (ContinuousIntegration p.9; CILab p.1). The unit tests are the verification core: they are fast — no database or file system — and focused, so a failure pinpoints the problem (ContinuousIntegration p.9). Finally the pipeline reports: the pull request shows a green check or a red failure, so the result is visible to anyone before the change is merged. If the build is red, the change does not get integrated until it is fixed; this shortens the gap between defect introduction and removal, which is the central mechanism by which CI reduces integration problems (ContinuousIntegration p.2, p.7). In effect the pipeline is an automated, impartial gatekeeper between my feature branches and the shared mainline.

Why & how — justification add-ons

- The pipeline runs on a neutral runner instead of my laptop because the build server is "the final authority on stability and quality", which removes the "it builds on my box" failure mode entirely (ContinuousIntegration p.6).
- Tests run inside the build rather than as a separate manual step because a self-testing build turns every integration into a verified integration — the definition of CI requires each integration to be "verified by an automated build" (ContinuousIntegration p.2, p.9).
- The visible pass/fail report on the pull request acts as the team's quality signal instead of private knowledge, because CI's value is collective: the team owns the code, not the individual (ContinuousIntegration p.5).
- Failures block the merge rather than being fixed "later", because the cost of a defect grows with the time between its introduction and its removal — the pipeline keeps that interval to minutes (ContinuousIntegration p.7).
- The pipeline could be extended with static analysis (Findbugs, PMD, Checkstyle) and coverage measurement (Cobertura) as automated quality steps, at the cost of CPU time — the course lists these as the natural next stages (ContinuousIntegration p.10, p.13–14).
- Dependency resolution is part of the verified path too: the workflow resolves shared jars from the registry on every run rather than from any developer's local cache, so a green build cannot silently depend on a jar that exists only on my machine (CILab p.1; ContinuousIntegration p.6).

Adapt to your lab

- Walk through [your actual workflow steps in order, copied from your YAML: trigger → checkout → JDK setup → Maven goal → test execution].
- Name [the Maven goal you run, e.g. `mvn clean install` or `mvn verify`] and [roughly how long a run takes].
- Describe [one concrete red build you experienced and what it caught], or state that all runs are green and what that demonstrates.

- If you added extras ([caching, static analysis, coverage, a badge in the README]), name them and mark them as your own additions beyond the lab minimum.

Key terms to drop

- **CI cycle** — commit → build → test → report, returning to development (ContinuousIntegration p.2).
- **The five CI principles** — environments based on stability; maintain a code repository; commit frequently and build every commit; make the build self-testing; keep the build fast (ContinuousIntegration p.3).
- **Source of record** — the repository, not any developer machine; disputes are settled by the build server (ContinuousIntegration p.6).
- **Unit vs system tests in CI** — unit: fast, focused, best for verifying builds; system: end-to-end, minutes to hours, too slow per-commit (ContinuousIntegration p.9).
- **Static code analysis** — finding common bugs and compliance issues without running the code: Findbugs, PMD, Checkstyle (ContinuousIntegration p.10, p.13).
- **Coverage** — measuring how much code the tests exercise: Cobertura, Emma (ContinuousIntegration p.10, p.14).
- **Hotspot (CI sense)** — an area of low testing and high complexity surfaced by unit-test analysis (ContinuousIntegration p.10).
- **Pull-request build** — the lab's trigger: build and test each pull request automatically with Maven (CILab p.1).

Why did you structure it that way?

Copy-paste answer

Why did I structure my CI pipeline that way? Because every structural choice in the pipeline — the pull-request trigger, the Maven build-and-test stages, and the merge gate — follows a CI principle. I structured the pipeline around three deliberate choices. First, **per-pull-request triggering instead of scheduled or manual builds**: this realises "commit frequently and build every commit" (ContinuousIntegration p.3, p.5). The benefit is causal traceability — when a build breaks, the cause is the one small change in that pull request, not a day's worth of merged work — and a shorter defect-introduction-to-removal gap (ContinuousIntegration p.7). Second, **a self-testing build with fast unit tests instead of a longer end-to-end suite in the gate**: unit tests are the best method for verifying builds because they are fast (no database or file system) and focused enough to pinpoint problems, and "keep the build fast" is itself a CI principle — a slow gate would tempt people to bypass it (ContinuousIntegration p.9, p.11). Third, **the pipeline as a merge gate rather than an after-the-fact report**: the build server, fed only from the repository, is the final authority on stability, which protects the shared mainline from "works on my machine" integration failures (ContinuousIntegration p.6). The deeper rationale is the maintenance perspective of the whole course: the Conclusion phase of every software change requires committing and integrating into a verified baseline (Conclusion p.2–3), and an automated pipeline makes that certification cheap enough to run on every single change instead of saving up risk for one big, painful integration (ContinuousIntegration p.2).

Why & how — justification add-ons

- I gated merges on the pipeline instead of trusting local test runs because a baseline must represent progress, not regression — automated verification of every integration is what keeps the shared code in a known-good state (Conclusion p.3; ContinuousIntegration p.2).

- I chose a simple linear workflow over a multi-stage deployment pipeline because the course project needs verified integration, not staged delivery; the principle of environments based on stability would only pay off with a real test/stage/production promotion path (ContinuousIntegration p.4).
- I kept heavy checks out of the per-commit path because static analysis costs "lots and lots of CPU", and the slide's own plea is "Please, KEEP IT FAST" — a fast gate gets used, a slow gate gets bypassed (ContinuousIntegration p.11).
- I let the workflow run the same Maven goals a developer runs locally instead of bespoke CI scripts, because identical commands mean a local green predicts a CI green, keeping feedback loops short (beyond slides — practical knowledge).
- I documented the pipeline in the repository so the structure is reviewable history, because pipeline setup is itself part of the graded maintenance work (What-To-Expect p.5).
- I kept exactly one authoritative workflow instead of per-developer build scripts, because a single shared definition is what makes the pipeline's verdict comparable across all branches and contributors — collective ownership applies to the build as much as to the code (ContinuousIntegration p.5–6).

Adapt to your lab

- Replace the three choices with [your actual structural decisions: trigger events, branch filters, job/step layout, test scope in the gate].
- Justify [each choice with its rejected alternative: e.g. push-trigger vs PR-trigger, full test suite vs unit tests only].
- If your pipeline has [multiple jobs or stages], explain the ordering and what each stage proves before the next runs.
- Tie one choice to [a real incident or constraint from your project: build time, a flaky test, a dependency issue with GitHub Packages].
- Name [one thing you deliberately left out of the pipeline] and why — exclusions are structural decisions too.

Key terms to drop

- **Commit frequently, build every commit** — the CI principle behind per-change triggering (ContinuousIntegration p.3, p.5).
- **Keep the build fast** — the principle constraining what belongs in the gate (ContinuousIntegration p.11).
- **Environments based on stability** — promote code to stricter environments as quality improves; the principle behind staged pipelines (ContinuousIntegration p.4).
- **Baseline** — the integrated, tested, agreed version the team builds on; CI makes baselining cheap and frequent (Conclusion p.2–3).
- **Regression protection** — a new baseline must never be worse than the previous one on already-working functionality (Conclusion p.3).
- **Integration hell** — the failure mode CI exists to kill: many changes merged late, breaking in combination (ContinuousIntegration p.2).
- **Team ownership** — "team owns the code, not the individual"; the social principle behind a shared gate (ContinuousIntegration p.5).

How and why did you refactor your code, and which refactoring techniques did you use?

Copy-paste answer

I refactored my JHotDraw code in the disciplined, catalog-driven way the course teaches. The *why*: refactoring improves the design of software, makes it easier to understand, helps find bugs, and ultimately helps me program faster, because bad design slows every future change (Refactoring1 p.4). In change-process terms I refactored at two points: **prefactoring** before implementing a change, to prepare a clean landing site so the new behaviour had one obvious place to live, and **postfactoring** afterwards, to remove the smells the change itself introduced (Refactoring1 p.5; ImpactAnalysis p.3). The *how*: I worked on a feature branch cut from the development branch (`git checkout -b your-feature development`), used SonarLint to flag smells relevant to my change request, applied one small catalog refactoring at a time, re-ran the tests after each step, and committed — a series of small behaviour-preserving transformations, never a big-bang rewrite, so the system stayed fully working throughout (RefactLab p.1). The techniques I used came from Fowler's catalog: Extract Method to break up long methods and name their intent (Refactoring1 p.11), Move Method to put behaviour next to the data it envies, Extract Class to split a class with more than one responsibility (Refactoring1 p.15–17), and Replace Conditional with Polymorphism to turn type-switching logic into subclass methods so new cases become new classes instead of edits to old code (Refactoring1 p.35). I verified behaviour preservation with the automated test suite, because without tests a refactoring is an unverified hope (BetterCode p.53–54).

Why & how — justification add-ons

- I refactored before adding function instead of bolting the feature onto messy code, because Fowler's trigger "refactor when you add function" turns a hard change into an easy one — the new code drops into a prepared structure (Refactoring1 p.5).
- I chose Extract Method over adding an explanatory comment because the method name documents the intent permanently, while a comment is a deodorant that drifts out of date (Refactoring1 p.8, p.11).
- I chose Replace Conditional with Polymorphism over extending the existing switch, because with polymorphism a new variant is a new subclass — existing, tested code stays closed to modification (Refactoring1 p.35; OOPrinciples p.6).
- I applied the Rule of Three instead of abstracting at the first duplication, because premature abstraction is its own smell (Speculative Generality) — the third occurrence is when a pattern is real and the abstraction pays (Refactoring1 p.5, p.7).
- I ran the test suite after every individual refactoring instead of once at the end, because each green run localises any behaviour change to the single move that caused it (RefactLab p.1; Software Testing p.81).
- I scoped my refactoring to the region of my change instead of sweeping the whole codebase, because refactoring exists to serve change — restructuring code that no change touches spends effort where no interest is being paid (Refactoring1 p.5; BeyondTechnicalDebt p.12).

Adapt to your lab

- Name [the refactorings you actually performed] on [the actual classes/methods in your fork].
- State [the smell each refactoring removed] and [the change request that made the smell matter].
- Reference [the feature branch and commits] holding the refactoring history.
- Quote [a before/after measure if you have one: method length, branch points, duplication removed].
- If you refactored toward a pattern ([Strategy / Template Method / Command]), say so and name Kerievsky's refactoring-to-patterns idea (HighLevelRefactoring p.16).

Key terms to drop

- **Prefactoring / Postfactoring** — behaviour-preserving restructuring before/after the actual change; same techniques, different position around Actualization (Refactoring1 p.5; ImpactAnalysis p.3).
- **Four reasons to refactor** — improves design; easier to understand; helps find bugs; helps you program faster (Refactoring1 p.4).
- **Extract Method** — turn a code fragment into a method whose name explains its purpose; the workhorse refactoring (Refactoring1 p.11).
- **Move Method** — relocate a method to the class whose features it uses most; the Feature Envy cure (Refactoring1 p.15).
- **Extract Class** — split one class doing the work of two; the Large Class / Divergent Change cure (Refactoring1 p.17).
- **Replace Conditional with Polymorphism** — move each leg of a type-conditional into an overriding subclass method (Refactoring1 p.35).
- **Small steps** — a sequence of small transformations, system kept fully working after each (RefactLab p.1).
- **Safety net** — the automated test suite that verifies behaviour preservation (BetterCode p.53–54).

How did you identify and fix code smells in your own code?

Copy-paste answer

I worked smell-by-smell, always tying the smell to my change request rather than cleaning at random (RefactLab p.1). Identification came from three sources. SonarLint in the IDE flagged candidate problems statically; reading the code during concept location surfaced structural symptoms the tool cannot judge, like Feature Envy and Divergent Change; and simple measurements caught size problems — methods over 15 lines, more than 4 branch points, 6-line duplicated blocks, parameter lists over 4 (Refactoring1 p.6–8; BetterCode p.10, p.16, p.23, p.25). For each candidate I asked the diagnostic question behind the smell: does this duplication force me to make the same fix in several places? Does this long method mix abstraction levels so I cannot name what it does? Does this switch mean a new case edits every dispatch site? Only smells that answered "yes" for my change got fixed — a smell is a symptom, not automatically a defect (Refactoring1 p.6). Fixing used the smell-to-refactoring mapping from the catalog: Duplicated Code → Extract Method (and Pull Up Method across siblings), Long Method → Extract Method, Large Class → Extract Class, Long Parameter List → Introduce Parameter Object, Switch Statements → Replace Conditional with Polymorphism (Refactoring1 p.6–8). Each fix was one small behaviour-preserving transformation followed by a test run and a commit on my feature branch (RefactLab p.1). The result is a documented trail in the repository: a smell named in the commit message, the catalog refactoring that removed it, and green tests proving behaviour was preserved.

Why & how — justification add-ons

- I fixed Duplicated Code with Extract Method instead of synchronising the copies by hand, because one shared method means one place to change forever, while synchronised copies fail the day someone forgets one (Refactoring1 p.6, p.11).
- I fixed the Long Parameter List with Introduce Parameter Object rather than reordering parameters, because the recurring parameter group was a missing domain concept — the new object names it and gives related behaviour a home (Refactoring1 p.6, p.38–41).
- I left some flagged items unfixed deliberately, because a smell that no current or foreseeable change touches accrues no interest — fixing it would be effort without benefit (Refactoring1 p.6;

BeyondTechnicalDebt p.12).

- I recorded the smell name in the commit message instead of a generic "cleanup", because the shared smell vocabulary makes the history reviewable — a reader can verify the diagnosis against the diff (Refactoring1 p.6; beyond slides — practical knowledge).
- I checked for new smells after actualizing my change (postfactoring) instead of stopping when it worked, because the change itself can introduce a too-long method or duplication, and leaving no smells behind after development work is the final maintainability guideline (BetterCode p.57–58).
- Where a smell sat directly in my landing zone, I fixed it before implementing the feature rather than after, because prefactoring localises the upcoming change and gives the new behaviour a clean place to drop into (Refactoring1 p.5).

Adapt to your lab

- Replace the generic mappings with [the 1–3 smells you actually found], [where], and [the exact refactorings applied].
- State [the SonarLint rule or measurement] that surfaced each one, or that you found it by reading.
- Link each fix to [your change request], showing the smell obstructed real work.
- Name [the commits] so the examiner can check the before/after.

Key terms to drop

- **Symptom, not bug** — a smell signals a deeper problem but the program may run perfectly (Refactoring1 p.6).
- **Duplicated Code** — same structure in multiple places; every fix must be repeated (Refactoring1 p.6).
- **Long Method / Large Class** — units doing too much at mixed abstraction levels (Refactoring1 p.6).
- **Feature Envy** — a method more interested in another class's data than its own; cure: Move Method (Refactoring1 p.6).
- **Shotgun Surgery vs Divergent Change** — one change touches many classes vs one class changes for many reasons (Refactoring1 p.6).
- **Introduce Parameter Object** — bundle parameters that travel together into a named object (Refactoring1 p.38–41).
- **G10 "leave no smells behind"** — after development work, no unit-level smells, dead code, magic constants, or bad comments remain (BetterCode p.57–58).
- **SIG thresholds** — 15 LOC per unit; ≤ 4 branch points; no ≥ 6 -line duplicates; ≤ 4 parameters (BetterCode p.10, p.16, p.23, p.25).

How did you apply software testing, why was testing important, and what kinds of tests did you use?

Copy-paste answer

I applied testing as the course's Verification practice: automated JUnit tests written against my JHotDraw changes, executed locally on every refactoring step and automatically in the CI pipeline on every pull request (TestLab1 p.1; CILab p.1). Testing was important for two reasons. Epistemically, it is the only evidence I have that a change is correct — while remembering Dijkstra's limit that testing shows the presence of bugs, never their absence, so a green suite is confidence, not proof (Software Testing p.6). Practically, the automated suite is what made the rest of my maintenance work *safe*: refactoring is only a behaviour-preserving transformation if something verifies the behaviour, and regression testing re-runs the whole suite after each change to catch

newly broken functionality within minutes (Software Testing p.81; Refactoring1 p.3). The kinds of tests I used: **unit tests** with JUnit 4 on the smallest testable units, designed black-box with equivalence partitioning (one representative per input class) and boundary-value analysis (testing at and around the edges, where off-by-one defects cluster — the lab explicitly requires boundary tests) (Software Testing p.8, p.11–12; TestLab1 p.1). I used **assertions** for representation invariants — checks for conditions that must always hold — keeping the rule that assertions are not error handling: expected errors get exceptions, broken invariants stop the program (Software Testing p.18; TestLab1 p.1). Where a unit depended on a collaborator, I isolated it with **test doubles**: stubs to feed canned data, mocks to verify interactions (Software Testing p.59–60). Finally, **acceptance-level scenarios** in JGiven expressed the user-visible behaviour as Given/When/Then (BDDLab p.1).

Why & how — justification add-ons

- I automated the tests instead of testing manually because manual tests are expensive and cannot run often, so only automation gives regression coverage on every change (Software Testing p.81).
- I designed tests with equivalence partitioning instead of inventing inputs ad hoc, because partitioning turns an infinite input space into a finite, justified test set — one representative per class the program treats the same (Software Testing p.12).
- I added boundary-value tests instead of testing only the happy middle, because defects cluster at the edges — empty, full, $B-1/B/B+1$ — and the lab requires exactly those cases (Software Testing p.11; TestLab1 p.1).
- I used a stub rather than a mock for pure queries, and a mock with `verify()` only when the interaction itself was the thing under test, because over-specifying interactions makes tests fragile (Software Testing p.59–60).
- I tested domain logic under the UI instead of driving the Swing GUI, because GUI-coupled tests are slow and fragile, while logic extracted below the UI is fast and deterministic to test (Software Testing p.29).
- I wrote a failing test before fixing a bug (TDD-style red/green/refactor), because the failing test pins the defect down and prevents it from returning (Software Testing p.35, p.39).

Adapt to your lab

- Name [the classes you wrote unit tests for] and [the boundary cases you covered, e.g. empty/full, min/max].
- State [the number or scope of your tests] and where they live in [your repo's test directory].
- If you used [Mockito stubs/mocks], name [the dependency you isolated] and why.
- Point at [a CI run where the tests executed automatically] (CILab p.1).
- If you wrote [JGiven scenarios], cross-reference your BDD answer.

Key terms to drop

- **Dijkstra's principle** — testing can demonstrate the presence of bugs, but not their absence (Software Testing p.6).
- **Unit / integration / system testing** — smallest unit in isolation; components across interfaces; whole system against the spec (Software Testing p.8).
- **Equivalence partitioning** — split inputs into classes treated the same; test one representative each (Software Testing p.12).
- **Boundary-value analysis** — test at the edges of classes, where off-by-one defects live (Software Testing p.11).
- **Black-box vs white-box** — tests from the spec versus tests from code structure and coverage (Software Testing p.8, p.13).

- **Regression testing** — re-run the suite after each change to catch newly broken behaviour (Software Testing p.81).
- **Test double / stub / mock** — stand-ins for dependencies: canned data vs interaction verification (Software Testing p.57–60).
- **Assertion vs exception** — assertions stop the program on broken invariants; exceptions handle expected errors and continue (Software Testing p.18; TestLab1 p.1).
- **JUnit** — the Java xUnit framework (by Gamma and Beck): `@Test`, `@Before`, assertion library (Software Testing p.51–53).
- **Red/green/refactor** — the TDD cycle: failing test, minimal pass, clean up (Software Testing p.39).

How did you improve the maintainability and readability of your code?

Copy-paste answer

I treated maintainability as a measurable property, not a feeling, and improved it along the course's two complementary tracks. At the **unit level**, I applied the Building Maintainable Software guidelines: I kept units short (the 15-line guideline), kept cyclomatic complexity down by limiting branch points to four per unit — extracting decision logic into named methods or replacing conditionals with polymorphism — eliminated duplicated blocks so each rule exists once, and kept parameter lists at four or fewer by introducing parameter objects (BetterCode p.10, p.16, p.23, p.25). At the **readability level**, I applied Clean Code practices: intention-revealing names so a reader does not decode abbreviations (CleanCode p.11), small functions doing one thing at one level of abstraction, ordered by the stepdown rule so the file reads top-down like an article (CleanCode p.24, p.53), and deleting noise comments in favour of code that explains itself (CleanCode p.38–44). I chose renaming over commenting because a good name removes the decoding cost permanently, while a comment drifts into a lie as the code changes. These two tracks reinforce each other and pay off in the change process itself: clear names make concept location faster, and low coupling and short units shrink the impact set of the next change — readable code is cheap-to-change code (ConceptLocation p.2; ImpactAnalysis p.5). The final discipline was the Boy Scout Rule: each time I touched a file for a change, I left it a little cleaner than I found it, so quality trends upward instead of decaying (CleanCode p.3).

Why & how — justification add-ons

- I shortened long units by Extract Method instead of adding section comments, because 15-line units are independently testable and nameable, while a commented 80-line method remains one untestable lump (BetterCode p.10; Refactoring1 p.11).
- I reduced branch points instead of just documenting the complex logic, because cyclomatic complexity approximates the minimum number of tests a unit needs — fewer branches means a verifiable unit (BetterCode p.16).
- I renamed identifiers to intention-revealing names instead of keeping short cryptic ones, because code is read far more often than it is written, so optimising for the reader pays back on every future visit (CleanCode p.11; Refactoring1 p.4).
- I applied the stepdown rule — callers above callees, one abstraction level per function — instead of arbitrary ordering, because a top-down file lets a reader grasp the flow without reading every detail (CleanCode p.24, p.53).
- I avoided train-wreck call chains per the Law of Demeter, because a client coupled to a deep object structure breaks whenever any intermediate changes (CleanCode p.64).
- I attended to module-level structure as well as unit-level polish, because maintainability is measured at every level — unit, module, component — and a clean method inside a heavily-coupled module still resists

change (BetterCode p.4, p.34).

Adapt to your lab

- Quote [one concrete before/after: a method shortened from N to M lines, a rename from `x` to `meaningfulName`].
- Name [the class where you applied the stepdown rule or removed noise comments].
- If you measured anything ([SonarLint issues count down, complexity down]), give the numbers.
- Tie one improvement to [a later change it made easier — the real proof of maintainability].
- Name [the guideline you most often violated before refactoring] — honesty about the starting point strengthens the improvement story.

Key terms to drop

- **Maintainability** — the degree to which software can be understood and safely changed (beyond slides — practical knowledge); the dominant cost of software lives here (SoftwareChange p.3; Introduction p.21–22).
- **The SIG guidelines** — write short units (15 LOC), simple units (≤ 4 branch points), code once (no ≥ 6 -line duplicates), small interfaces (≤ 4 parameters), plus automated tests and clean code (BetterCode p.10, p.16, p.23, p.25, p.53, p.57).
- **Intention-revealing name** — a name that says why a thing exists, what it does, how it is used (CleanCode p.11).
- **Stepdown rule** — code reads top to bottom, descending one abstraction level at a time (CleanCode p.24).
- **Newspaper metaphor** — headline at the top, detail below (CleanCode p.53).
- **Boy Scout Rule** — always leave the code cleaner than you found it (CleanCode p.3).
- **Law of Demeter** — talk only to immediate friends; no `a.getB().getC()` chains (CleanCode p.64).
- **DRY** — Don't Repeat Yourself; duplication "may be the root of all evil" (CleanCode p.33).

Which Clean Code concepts did you apply, and what decisions did you take?

Copy-paste answer

From the Clean Code book I applied four concept families to my own code, each as a deliberate decision. **Names:** I chose intention-revealing, pronounceable, searchable names and avoided disinformation and type-encoding — renaming was my single most frequent edit, because a name is the cheapest documentation that cannot drift (CleanCode p.11–16). **Functions:** I made functions small, doing one thing, at one level of abstraction, with as few arguments as possible — preferring one argument over two, bundling related arguments into objects, and eliminating flag arguments by splitting the function, because a boolean parameter is an admission that the function does two things (CleanCode p.24, p.27–30). I separated commands from queries so a method either changes state or answers a question, never both, which makes queries safe to call anywhere (CleanCode p.32). **Comments:** I deleted redundant, mumbling, and journal comments and replaced explanatory comments with extracted, well-named methods, keeping only comments that add information code cannot carry (CleanCode p.38–44). **Tests:** I followed the F.I.R.S.T. properties — fast, independent, repeatable, self-validating, timely — so the suite stays trustworthy enough to run constantly (CleanCode p.77). The unifying decision was the Single Responsibility Principle: one reason to change per class, which I enforced by extracting classes whenever a second responsibility crept in (CleanCode p.79). Throughout, the Boy Scout Rule governed the cadence: every visit to a file paid a small cleanliness dividend (CleanCode p.3). Each of these is visible in my repository as commits accompanying the functional changes.

Why & how — justification add-ons

- I split a function with a boolean parameter into two named functions instead of keeping the flag, because flag arguments force every call site to be decoded and signal the function does more than one thing (CleanCode p.29).
- I converted an argument cluster into an argument object instead of a long signature, because the cluster was a hidden concept — naming it shortened every call and gave related behaviour a home (CleanCode p.31).
- I separated a method that both set state and returned success into a command and a query, because callers of mixed methods get side effects they did not ask for (CleanCode p.32).
- I deleted the commented-out code instead of keeping it "just in case", because version control already preserves history, and dead code in comments only scares readers (CleanCode p.38–44; Git Lab p.3).
- I kept tests to one concept each and made them fast and order-independent, because a slow or flaky suite stops being run, and an unrun suite protects nothing (CleanCode p.77).
- I formatted consistently with the project's existing conventions instead of my personal style, because team rules beat individual preference — a codebase with one voice is cheaper to read than a patchwork of styles (CleanCode p.60).

Adapt to your lab

- Name [2–3 concrete renames] and [the function you split or shrank], with [the class] and [the commit].
- Quote [one deleted comment and the method name that replaced it].
- State [where you enforced SRP: the class you extracted and the responsibility you moved].
- If you applied [the Three Laws of TDD] anywhere, name the test class that grew alongside the code (CleanCode p.74).

Key terms to drop

- **Intention-revealing names / disinformation** — names that explain purpose; never names that mislead (CleanCode p.11–12).
- **Small functions, one thing** — a function should do one thing, at one level of abstraction (CleanCode p.24).
- **Niladic/monadic/dyadic** — argument counts; fewer is better, zero is ideal (CleanCode p.27–30).
- **Flag argument** — a boolean selecting behaviour; split the function instead (CleanCode p.29).
- **Command/Query Separation** — do something or answer something, never both (CleanCode p.32).
- **Bad-comment taxonomy** — mumbling, redundant, mandated, journal, noise comments — delete or replace with code (CleanCode p.38–44).
- **F.I.R.S.T.** — fast, independent, repeatable, self-validating, timely tests (CleanCode p.77).
- **SRP** — one reason to change per class (CleanCode p.79).
- **Boy Scout Rule** — leave it cleaner than you found it (CleanCode p.3).

Explain your architecture decisions (clean architecture)

Copy-paste answer

My architectural decisions follow Robert C. Martin's Clean Architecture, whose core is the **Dependency Rule**: source-code dependencies point only inward, toward stable abstractions (Clean Architecture p.5, p.8). The architecture is four concentric layers — Entities (enterprise business rules) innermost, then Use Cases

(application business rules), then Interface Adapters (controllers, presenters, gateways), then Frameworks & Drivers (UI, database, web) outermost (Clean Architecture p.7). The inner layers know nothing about the outer ones; the UI and the database are "details" — plug-ins to the business rules, not the centre of the system (Clean Architecture p.16–18). I applied this thinking to my project in two ways. First, in analysing JHotDraw: the framework separates the drawing model (figures and their rules) from the Swing presentation, which is exactly the separation that lets the domain be tested and changed independently of the GUI — the model plays the role of the inner layers, Swing the role of Frameworks & Drivers. Second, in my own changes I kept business logic out of UI classes and behind interfaces: where a component needed a collaborator, I depended on an abstraction and let the concrete implementation be supplied from outside, which is the Dependency Inversion Principle — the mechanism by which the Dependency Rule is achieved at class scale (OOPrinciples p.12). The benefit is bounded change: swapping a detail (a storage format, a UI control) actualizes only in the outer layer, and the ripple stops at the boundary interface. The architecture's four promised characteristics — testable, independent of UI, independent of database, independent of frameworks — are all consequences of that one rule (Clean Architecture p.3).

Why & how — justification add-ons

- I put the interface next to the business rules and its implementation in the detail layer, instead of letting the rules call the detail directly, because that inversion means the database can be swapped without touching the use cases — "the database is a detail" (Clean Architecture p.16–18).
- I kept the view logic-free instead of putting decisions in UI handlers, because a view "so stupid you don't test it" moves all testable logic into presenters and use cases where unit tests reach it (Clean Architecture p.15).
- I distinguished architecture from design patterns deliberately: patterns like Observer solve a local collaboration problem, while an architecture like Clean Architecture governs the whole system's dependency structure (Clean Architecture p.2).
- I respected that data flows outward and back through the layers while dependencies still point only inward, because confusing data flow with dependency direction is what reintroduces coupling to details (Clean Architecture p.8).
- I justified the layering economically: stable things (business rules) must not depend on volatile things (frameworks), because a dependency on something volatile inherits its rate of change (Clean Architecture p.8; OOPrinciples p.12).
- I used the SOLID principles as the class-scale enforcement of the architecture, because SRP, OCP, LSP, ISP and DIP are precisely what keep each layer cohesive and the boundaries between layers real (OOPrinciples p.3–13).

Adapt to your lab

- Name [the place in your JHotDraw work where domain and presentation are separated, e.g. figure model vs Swing view classes].
- Describe [one interface you introduced or used] and [the concrete implementation behind it].
- If you prepared the Itslearning clean-architecture diagram, attach/redraw it and label [the four layers] with classes from your project (What-To-Expect p.4).
- State [one change that the layering made cheap — or would make cheap — in your codebase].
- Name [one SOLID principle visible in your own change] and the class pair it governs (OOPrinciples p.3–13).

Key terms to drop

- **Dependency Rule** — source-code dependencies point only inward toward stable abstractions (Clean Architecture p.8).
- **The four layers** — Entities, Use Cases, Interface Adapters, Frameworks & Drivers (Clean Architecture p.7).
- **Entities / Use Cases** — enterprise-wide rules; application-specific orchestration of the entities (Clean Architecture p.7, p.9).
- **"The database is a detail"** — DB, UI, frameworks are plug-ins to the business rules (Clean Architecture p.16–18).
- **Entity Gateway / Repository** — the interface declared with the business rules, implemented in the detail layer (Clean Architecture p.18).
- **Boundary** — the interface through which data enters and leaves a use case (Clean Architecture p.9).
- **Architecture vs design pattern** — system-wide structure versus a local reusable solution (Clean Architecture p.2).
- **Four characteristics** — testable; independent of UI; independent of database; independent of frameworks (Clean Architecture p.3).
- **DIP** — depend on abstractions, not concretions; the Dependency Rule at class scale (OOPrinciples p.12).

Walk through one technical decision and justify it

Copy-paste answer

A representative decision from my project: while implementing my change, I had to dispatch behaviour that varied by figure type, and I had two options — extend an existing conditional, or introduce polymorphism.

The decision: I replaced the type-conditional with polymorphism, moving each branch into an overriding method on the corresponding subclass (Refactoring1 p.35). **The rejected alternative:** adding another case to the existing if/switch chain. That would have been faster to type, but it concentrates every variant's logic in one growing method and means each new variant edits already-tested code — exactly the Switch Statements smell (Refactoring1 p.7). **The benefits:** first, the design now follows the Open/Closed Principle — open for extension, closed for modification — so the next variant is a new subclass and zero edits to existing code (OOPrinciples p.6). Second, the change propagation of future variants shrinks: the impact set of "add a type" becomes one new class instead of every dispatch site, which is the architectural lever the Drawlets case quantifies — refactoring roughly halved the classes a change touched while the line count barely moved (Drawlets p.31, p.37). Third, each subclass's behaviour is now independently unit-testable. **The cost I accepted:** more classes and a level of indirection, which I judged worthwhile because this axis of the design demonstrably varies — the course's two impact-analysis criteria, required effort versus clarity of the resulting code, pull in opposite directions here, and I chose clarity because this code will be changed again (ImpactAnalysis p.15). The decision is visible in my repository as a commit pair: the prefactoring commit restructuring the conditional, then the actualization commit adding the new behaviour.

Why & how — justification add-ons

- I weighed effort against clarity explicitly, because the course names these as the two criteria for choosing where and how to make a change, and they genuinely conflict — the cheap patch is rarely the clear one (ImpactAnalysis p.15).
- I preferred the structure that shortens future change propagation, because the Drawlets data shows the true cost of a change is the number of classes the change ripples through, not the lines edited (Drawlets

p.30, p.37).

- I made the restructuring a separate, behaviour-preserving commit before the functional change, because separating prefactoring from actualization keeps each commit verifiable on its own — tests prove the first changed nothing and the second changed exactly one thing (Refactoring1 p.5; ImpactAnalysis p.3).
- I depended on an abstraction at the decision point because DIP is how you adhere to OCP — the abstraction is the hinge where the design bends without being modified (OOPrinciples p.12; DesignPrinciplesAndPatterns p.13).
- I documented the rejected alternative in the report deliberately, because the examiner grades the justification — why and how — not just the outcome (What-To-Expect p.1, p.4).
- I validated the decision with tests on both sides — green before the restructuring and green after the new behaviour — because a justification without verification is just an opinion (Software Testing p.81).

Adapt to your lab

- Swap in [your real decision: a refactoring choice, a test-design choice, a pipeline-structure choice, a data-structure choice].
- Name [the rejected alternative you genuinely considered] — the pattern requires it.
- List [two or three concrete benefits in maintenance vocabulary: readability, smaller impact set, testability, OCP].
- State [the cost you accepted] and why it was worth it — acknowledging the trade-off earns credibility.
- Point at [the commit(s)] embodying the decision.

Key terms to drop

- **Effort vs clarity** — the two impact-analysis criteria for choosing among candidate change locations; they often conflict (ImpactAnalysis p.15).
- **Open/Closed Principle** — open for extension, closed for modification (OOPrinciples p.6).
- **Change propagation** — the spread of secondary changes through dependencies; the real cost driver of a change (Drawlets p.37; Actualization p.16–22).
- **Hinge point** — Martin's metaphor for an abstraction: where the design bends without being modified (DesignPrinciplesAndPatterns p.13).
- **Prefactoring commit** — a behaviour-preserving restructuring commit preceding the functional change (Refactoring1 p.5).
- **Trade-off** — every design choice buys a benefit at a cost; naming both is what makes a justification honest (ImpactAnalysis p.15).
- **Ripple effect** — a change spreading to other components via dependencies; what good structure limits (Actualization p.5).

How did you use version control, and how did you structure your commits and branches?

Copy-paste answer

I used Git, a decentralized version-control system, with GitHub hosting my fork of JHotDraw (Git Lab p.7, p.9; IntroLab p.1). Version control gave the project its essential guarantees: every version archived, full history kept, collaboration coordinated, discipline enforced, and recovery always possible (Git Lab p.3). I worked with Git's four areas deliberately — workspace, staging index, local repository, remote — staging and committing locally, then pushing to the remote, because in a decentralized VCS `commit` records only to my local history

and `push` is the separate act of sharing (Git Lab p.11). Before every commit I ran `git status` and `git diff` so I knew exactly which files and lines I was recording, which kept stray edits and debug code out of history (Git Lab p.13). **Branch structure:** I followed GitHub flow — each piece of work on its own feature branch, cut from the development line (`git checkout -b your-feature development`), keeping work-in-progress isolated from the baseline until it was ready to integrate via a pull request that triggered the CI build (IntroLab p.1; RefactLab p.1; CILab p.1). **Commit structure:** small, functional commits, each one coherent step — a single refactoring, a test added, one piece of a feature — with messages naming the intent (the smell removed, the phase of the change), because small commits are what CI's "commit frequently and build every commit" principle assumes, and they make the history reviewable change by change (ContinuousIntegration p.5). I also used GitHub Issues to record change requests — filing an Issue was the course's first Initiation step, tying repository work to the change process (Git Lab p.19).

Why & how — justification add-ons

- I committed in small functional steps instead of one large end-of-day commit, because small commits localise any failure the pipeline finds to one reviewable diff and keep merges trivial (ContinuousIntegration p.5).
- I branched per feature instead of working on the mainline, because isolation means my half-done work can never break the baseline others build on (IntroLab p.1; Conclusion p.2–3).
- I wrote commit messages naming the maintenance action ("postfactoring: extract duplicated move logic") instead of "fixes", because the repository is itself graded and the message is where the why/how lives in the history (What-To-Expect p.5; beyond slides — practical knowledge).
- I never committed build output or binaries, only source, because generated artifacts do not diff, bloat history forever, and are regenerated by `mvn clean install` anyway (Git Lab p.6 area guidance; IntroLab p.1).
- I pulled (fetch + merge) before pushing instead of force-pushing over teammates' history, because integrating others' changes locally first is how conflicts get resolved by the person who understands both sides (Git Lab p.11).
- I treated the history as documentation for my future self, because in maintenance the most common reader of a diff is the next person to change that code — often me, months later, having forgotten everything (Git Lab p.3; Introduction p.9).

Adapt to your lab

- Name [your actual branches: feature/refactoring branches and the development/main lines] and the merge mechanism ([pull requests]).
- Quote [2–3 real commit messages] that show the maintenance vocabulary.
- State [the Issue(s) you filed] as change requests and which commits closed them (Git Lab p.19).
- Give [a rough commit count or cadence] as evidence of small-step work.

Key terms to drop

- **Decentralized VCS** — every clone is a complete repository with full history; commits are local, pushing shares them (Git Lab p.7, p.9).
- **The four areas** — workspace → index/stage → local repository (HEAD) → remote (origin) (Git Lab p.11).
- **commit vs push** — record locally versus publish to the team (Git Lab p.11).
- **pull = fetch + merge** — download remote commits, then integrate them into the workspace (Git Lab p.11).

- **GitHub flow / feature branch** — per-feature branches keeping concurrent work isolated until integration (IntroLab p.1).
- **Fork** — a server-side copy under your own account, enabling pushes without write access to the original (IntroLab p.1; Git Lab p.18–19).
- **Issue** — GitHub's change-request record; filing one is an Initiation act (Git Lab p.19).
- **Baseline** — the integrated, tested shared state that branches are cut from and merged back into (Conclusion p.2–3).

What maintenance work does your repository show?

Copy-paste answer

My repository is a portfolio of the full software-maintenance lifecycle, mapped to the course labs. **Initiation:** change requests recorded as GitHub Issues and user stories ("As a [user type], I want [goal] so that [reason]"), the course's entry point into the change process (Git Lab p.19; BDDLab p.1). **Concept location and impact analysis:** the analysis lab's deliverable — the table of packages visited after locating the concept, with class counts and comments on what each package contributes to my feature — documents how I found and scoped my change (AnalysisLab p.1–2). **Refactoring:** a feature branch with smell-driven, behaviour-preserving commits — SonarLint findings addressed with named catalog refactorings (RefactLab p.1). **Testing:** JUnit 4 unit tests with boundary-value cases as the testing lab requires, plus JGiven Given/When/Then scenarios mapping my user stories to executable acceptance criteria with AssertJ assertions (TestLab1 p.1; BDDLab p.1). **Pipeline setup:** the GitHub Actions workflow under `.github/workflows/` building each pull request with Maven and executing tests automatically (CILab p.1). **Overall maintenance discipline:** small functional commits, feature branches integrated through pull requests, no build output committed, and a history whose messages narrate the change phases. Together these are exactly the items the exam evaluates in the repository — code changes, refactoring work, testing, pipeline setup, and overall maintenance work (What-To-Expect p.5). The repository is therefore not just code but evidence: each claim in this report corresponds to a branch, commit, workflow run, or Issue that can be inspected.

Why & how — justification add-ons

- I kept the lab artifacts in the repository rather than separate documents where possible, because co-locating evidence with code makes the maintenance story verifiable from the history alone (What-To-Expect p.5; beyond slides — practical knowledge).
- I made each lab a separate branch/commit cluster instead of one merged blob, because separable units of work let an examiner (or teammate) review each maintenance activity on its own terms (ContinuousIntegration p.5).
- I documented the impact-analysis package walk in the portfolio table because an impact set is a prediction, and recording it is what later allows the prediction to be checked against what actually changed (AnalysisLab p.1–2; Actualization p.23).
- I kept the pipeline green at all times, because a red mainline would mean the repository's baseline is not trustworthy — the cardinal sin of integration discipline (Conclusion p.3; ContinuousIntegration p.6).
- I linked report claims to repository artifacts in both directions, because the report is graded on reflecting the work and the repository on containing it — unexplained work and unevidenced claims both leak marks (What-To-Expect p.1, p.5).

Adapt to your lab

- Replace the generic inventory with [your actual branch names, Issue numbers, workflow file name, test class names].
- State [which labs you completed] and where each one's evidence lives.
- Name [the single best showcase commit or PR] for each of: refactoring, testing, pipeline.
- If anything is incomplete, [name it and what you would do next] — an honest maintenance backlog is itself maintenance thinking.

Key terms to drop

- **Portfolio** — the accumulated lab work the course assesses; the repository is its primary form (Introduction p.5–6).
- **Change request / Issue** — the recorded trigger of every change (Git Lab p.19).
- **User story** — "As a [user type], I want [goal] so that [reason]" (BDDLab p.1).
- **Estimated impact set** — the documented prediction of which classes a change touches (AnalysisLab p.1; ImpactAnalysis p.3–4).
- **Feature branch** — the isolation unit of each piece of maintenance work (RefactLab p.1; IntroLab p.1).
- **Self-testing pipeline** — the workflow that builds and tests every pull request (CILab p.1).
- **Living documentation** — JGiven reports regenerated from each test run, documentation that cannot go stale (BDD p.4, p.17–18).
- **Baseline integrity** — the shared state must always be known-good (Conclusion p.3).

How did you locate the code you needed to change (concept location)?

Copy-paste answer

Concept location "finds the code snippet where a change is to be made" (ConceptLocation p.2) — the bridge between a change request written in domain vocabulary and the class where work must start. I followed the course's method. First I **extracted the concepts** from my change request: the domain nouns and verbs that might correspond to code (ConceptLocation p.6). Then I **classified** them the way the Drawlets worked example does — irrelevant (ordinary request words with no code counterpart), external (things the change will introduce, which cannot be searched for because they do not exist yet), and significant (concepts already implemented in the design) — because only significant concepts are valid anchors for a search (Drawlets p.7–8). Then I **searched** for the significant concepts using the course's technique families: human knowledge and documentation first, then dynamic search — running JHotDraw under the IDE debugger to see which classes actually participate in the feature — and static search over the class dependency graph, following dependencies from a class I already understood toward the concept, with grep-style pattern matching over identifiers as a shortcut (ConceptLocation p.7). I accepted **partial comprehension** as the strategy: in a framework of hundreds of classes you aim for the minimum essential understanding via an as-needed strategy, like learning one route through a large city rather than every street (ConceptLocation p.3). The search is genuinely fallible — the Drawlets trace shows a wrong turn into the container class before backtracking to the right abstraction — so I treated dead ends as part of the algorithm, backtracking and re-selecting rather than guessing (Drawlets p.9–15). The located class became the seed of my impact analysis (ImpactAnalysis p.3–4).

Why & how — justification add-ons

- I used the IDE debugger (dynamic search) instead of only reading code, because an execution trace shows which classes *actually* run for the feature, cutting through naming mismatches between the request's vocabulary and the code's (ConceptLocation p.7).
- I started the dependency search from classes I already comprehended instead of opening files at random, because the search algorithm expands from understood anchors along dependency edges, asking at each module whether the concept is implemented locally (ConceptLocation p.10).
- I checked "is the concept implemented in this module itself?" before descending into suppliers, because the stop condition is local implementation — asking the composite question first would always send me past the very class I should change (ConceptLocation p.10).
- I classified request concepts before searching, because searching for external or irrelevant concepts wastes effort — you can only locate what already exists (Drawlets p.8).
- I recorded my search path including dead ends, because the backtrack trail documents *why* the located class is the right one, not just *that* I found it (Drawlets p.9–15).
- I distinguished a concept's name from its intension, because the code rarely uses the request's word — recognising the concept inside a class named something else entirely is reasoning from meaning, not string matching (ConceptLocation p.4–5).

Adapt to your lab

- Name [your change request] and [the concepts you extracted from it].
- State [which were significant vs external vs irrelevant] in your classification.
- Describe [the actual search: where you started, the debugger session or grep terms, one dead end if you had one].
- Name [the class/method you located] — the starting point of your change.
- State [roughly how long location took] and what made it fast or slow — the as-needed strategy is measured in your time (ConceptLocation p.3).

Key terms to drop

- **Concept location** — finding the code snippet where a change is to be made (ConceptLocation p.2).
- **Vocabulary gap** — requests speak domain language; code scatters and renames those concepts (ConceptLocation p.2).
- **Concept extraction / classification** — list the request's concepts; sort into irrelevant / external / significant (Drawlets p.7–8).
- **Partial comprehension / as-needed strategy** — aim for minimum essential understanding, not whole-system mastery (ConceptLocation p.3).
- **Dynamic vs static search** — execution traces and the debugger versus dependency search and grep over the code (ConceptLocation p.7).
- **Dependency search** — walk the class dependency graph from understood anchors, asking the local-implementation question at each step (ConceptLocation p.10).
- **Backtracking** — abandoning a dead-end candidate and re-selecting; error is part of the algorithm (Drawlets p.9–15).
- **Initial impact set** — the located classes, handed to Impact Analysis as its seed (ImpactAnalysis p.3–4).

How did you assess the impact of your change (impact analysis)?

Copy-paste answer

Impact analysis is "identifying the potential consequences of a change, or estimating what needs to be modified to accomplish a change" (AnalysisLab p.1) — it grows the single located class into the full set the change will touch, so cost and risk are understood before any code is edited. I ran it as the course's marking algorithm over the interaction graph, following Figure 7.9 in Rajlich (AnalysisLab p.1; ImpactAnalysis p.11–12). Starting from my concept-location class marked CHANGED, I inspected its neighbours — every class one interaction away — marking each as CHANGED if it needs modification, UNCHANGED/INSPECTED if it does not, or PROPAGATING if it needs no edit itself but relays the change between its neighbours, in which case its neighbours still join the inspection queue (ImpactAnalysis p.10–12). The loop ends when no classes remain scheduled; everything marked CHANGED or PROPAGATING is the **estimated impact set** (ImpactAnalysis p.3–4). I used both static analysis (the dependency structure) and dynamic analysis (running the feature) to find interaction edges, as the lab requires, because interactions include not only call dependencies but coordinations that a dependency-only view misses (AnalysisLab p.1; ImpactAnalysis p.5). I documented the walk as the lab's portfolio table — each package visited after concept location, the number of classes, and what I learned about its contribution to my feature (AnalysisLab p.1–2). I treated the result as a *prediction*: actualization is the moment of truth that confirms or refutes it, and industrial data shows estimates miss badly — Ericsson's programmers achieved 100% precision but only 32% recall, missing two-thirds of the classes that actually changed, because dependencies are invisible (Actualization p.23, p.26–28).

Why & how — justification add-ons

- I expanded through PROPAGATING classes instead of stopping at unchanged ones, because a class can relay change without needing edits itself — marking it UNCHANGED would wrongly cut off the far side of the graph (ImpactAnalysis p.10, p.12).
- I walked interactions in both directions instead of following call arrows one way, because change propagates from callee to caller as well — a changed interface breaks its clients (ImpactAnalysis p.5, p.9).
- I combined static and dynamic analysis instead of trusting the dependency graph alone, because runtime coordination (two classes wired together inside a third) creates impact paths no static arrow shows (AnalysisLab p.1; ImpactAnalysis p.5, p.8).
- I sized the change by the impact set rather than by expected lines of code, because the Drawlets data shows propagation size, not edit size, is the true cost of a change — 0.5% of the lines still meant 13 modified classes (Drawlets p.30, p.37).
- I kept the estimate humble and re-checked during actualization, because under-estimation is systemic — invisible dependencies make recall, not precision, the hard metric (Actualization p.27–28).
- I recorded what I learned about each visited package, not just its name, because the comments column is what turns the marking walk into reusable comprehension for the next change in the same region (AnalysisLab p.1–2).

Adapt to your lab

- Name [your seed class from concept location] and [the packages/classes in your estimated impact set].
- Reproduce or reference [your portfolio Table 1: package name / number of classes / comments] (AnalysisLab p.2).
- State [one PROPAGATING-style class you found — visited, unchanged, but relaying impact], if you had one.

- Compare [your estimate vs what actually changed] — your personal precision/recall — and what surprised you.
- If you used [JRipples or the IDE's dependency views] to support the walk, name the tool and what it automated (BBPR05).

Key terms to drop

- **Impact analysis** — identifying a change's potential consequences / estimating what must be modified (AnalysisLab p.1).
- **Initial vs estimated impact set** — the concept-location seed versus the full expanded prediction (ImpactAnalysis p.3–4).
- **Interaction graph** — classes as nodes, interactions (dependency or coordination) as edges the analysis walks (ImpactAnalysis p.5–6).
- **Marks** — BLANK / NEXT / CHANGED / UNCHANGED / PROPAGATING: the bookkeeping that drives and terminates the loop (ImpactAnalysis p.11–12).
- **PROPAGATING ("mailman") class** — needs no edit but relays change between neighbours (ImpactAnalysis p.10).
- **JRipples** — the tool supporting incremental impact-set marking (BBPR05).
- **Precision / recall** — predicted-and-right over predicted; predicted-and-right over actually-changed — Ericsson: 100% / 32% (Actualization p.26–27).
- **Moment of truth** — change propagation during actualization confirms or refutes the impact estimate (Actualization p.23).

Walk through one complete software change from start to finish

Copy-paste answer

I will narrate one change through all phases of the course's change process: Initiation → Concept Location → Impact Analysis → Prefactoring → Actualization → Postfactoring → Conclusion, with Verification running across the implementation phases (ImpactAnalysis p.3). **Initiation:** the change began as a recorded request — a GitHub Issue with a user story stating what was wanted and why it was worth doing (Git Lab p.19; BDDLab p.1). **Concept location:** I extracted the request's concepts, classified them, and searched the code — debugger trace plus dependency search from classes I understood — until I found the class that locally implements the concept; that class is the change's starting point (ConceptLocation p.2, p.7, p.10). **Impact analysis:** I expanded that seed across the interaction graph with the marking algorithm into an estimated impact set, recorded as the package/class table (ImpactAnalysis p.11–12; AnalysisLab p.1–2). **Prefactoring:** before adding behaviour, I refactored the change region — behaviour-preserving only — so the new code would have one clean place to land (Refactoring1 p.5). **Actualization:** I implemented the new functionality, incorporated it into the old code, and propagated the change — visiting each neighbour of every edited class, restoring consistency until no inconsistencies remained (Actualization p.1, p.16–22). **Postfactoring:** I removed the smells the change introduced — the method that had grown too long, the small duplication — leaving the code no worse than I found it (Refactoring1 p.5; BetterCode p.57). **Conclusion:** I committed to the repository, opened the pull request, and let the CI pipeline build and test the integration into a new baseline (Conclusion p.2–3; CILab p.1). **Verification** ran throughout: unit tests after every refactoring step, boundary tests on the new logic, and the JGiven scenario confirming the original acceptance criterion, so "done" meant demonstrated, not assumed (Software Testing p.81; BDDLab p.1).

Why & how — justification add-ons

- I followed the phased process instead of "just coding", because each phase consumes the previous one's output — location feeds the impact set, the impact set scopes the refactoring and implementation — so nothing is done blind (ImpactAnalysis p.3).
- I separated prefactoring from actualization instead of restructuring and implementing in one commit, because behaviour-preserving and behaviour-changing edits need different verification: tests must stay green through the first and change deliberately in the second (Refactoring1 p.3, p.5).
- I propagated the change systematically through neighbours instead of stopping when "it worked", because a change makes neighbouring classes inconsistent, and unvisited inconsistency is exactly how regressions ship (Actualization p.16–22).
- I closed the change with an integrated, pipeline-verified baseline instead of leaving it on my branch, because a change that never reaches the shared, certified state has delivered nothing (Conclusion p.2–3).
- The course's Drawlets example shows why this discipline pays: a 91-line change (0.5% of the code) still propagated through about 13 classes — the process is what kept all 13 consistent (Drawlets p.30, p.37).
- I kept Verification continuous instead of saving testing for the end, because the process places it alongside every implementation phase — each phase's output was checked while its context was still in my head (ImpactAnalysis p.3; Software Testing p.81).

Adapt to your lab

- Swap in [your actual change: the Issue/user story, the located class, the impact-set packages, the refactorings before and after, the commits and PR].
- Give [one concrete number: classes in your impact set, tests added, commits in the branch].
- Name [the phase that surprised you most and why] — reflection is the report's currency.
- If a phase was trivial in your case ([no prefactoring needed]), say so and justify why.
- Quote [the user story or Issue text verbatim] at the start of the walkthrough — it frames every later phase.

Key terms to drop

- **The phase sequence** — Initiation, Concept Location, Impact Analysis, Prefactoring, Actualization, Postfactoring, Conclusion, with Verification alongside (ImpactAnalysis p.3).
- **Initiation** — the change request captured, prioritised, accepted (Git Lab p.19; BDDLab p.1).
- **Actualization's three jobs** — implement new functionality, incorporate it, propagate secondary changes (Actualization p.1).
- **Change propagation** — restoring consistency neighbour by neighbour until no marks remain (Actualization p.16–22).
- **Conclusion's three steps** — commit (to the team), new baseline (certified for the project), release (to users) (Conclusion p.2–3).
- **Verification** — the cross-cutting correctness guarantee: tests, acceptance scenarios, no regressions (Software Testing p.81).
- **Drawlets numbers** — 91 production LOC (0.5%) changed, ~13 classes + 2 new; refactoring later cut classes touched to 5 (Drawlets p.30, p.37).

What is technical debt, and where did you encounter it in your project?

Copy-paste answer

Technical debt is "stuff that isn't supposed to be there and is in the way of the stuff that is supposed to be there" (BeyondTechnicalDebt p.2, quoting Ford, Parsons and Kua) — substandard code framed as a loan: the **principal** is the substandard code itself, measured as complexity, and the **interest rate** is how often you must work with it, measured as change frequency in version control (BeyondTechnicalDebt p.12). The framing matters because it makes the cost rational to manage: complexity that is never touched costs nothing, while complicated code you change often — a **hotspot** — bleeds effort on every visit and is the rational first repayment target (BeyondTechnicalDebt p.12). Debt is also structurally inevitable: Lehman's laws say a useful system must continually change or become progressively less satisfactory, and its complexity increases unless work is done to reduce it (BeyondTechnicalDebt p.3). I encountered debt in my project in exactly these terms. The code I worked on contained inherited debt — long, complex methods and duplication in the region of my change — which I experienced as interest: my concept location took longer, my impact set grew, and my change had to thread through structure that resisted it. I also *created* debt knowingly at one point, taking a shortcut to get behaviour working, and repaid it in postfactoring — which is the honest cycle the metaphor describes: borrowing is legitimate if the interest is visible and the repayment scheduled (BeyondTechnicalDebt p.12; Refactoring1 p.5). What I learned from the course's behavioural-analysis perspective is that a static snapshot of violations cannot prioritise repayment; only combining complexity with change frequency — how the organization actually works with the code — says where to start (BeyondTechnicalDebt p.7–10).

Why & how — justification add-ons

- I prioritised cleanup by hotspot (complexity × change frequency) instead of fixing the "worst" code globally, because static analysis can never say whether excess complexity actually matters — only change history shows which debt charges interest (BeyondTechnicalDebt p.10, p.12).
- I budgeted refactoring as recurring work instead of a one-off cleanup, because Lehman's laws make complexity growth the default trajectory — counteracting it is maintenance, not a special event (BeyondTechnicalDebt p.3).
- I watched for complexity rising while size stays flat, because that divergence is the signature of code rotting in place — the course's `checker.ts` example grew to enormous complexity on flat LOC (BeyondTechnicalDebt p.15).
- I treated unowned, poorly understood code as debt too, because legacy is partly a people problem — code whose knowledge has left is risky to change regardless of its intrinsic quality (BeyondTechnicalDebt p.20–21).
- I logged the shortcut I took instead of hiding it, because invisible debt cannot be negotiated with — the loan metaphor only works when the borrowing is on the books (BeyondTechnicalDebt p.2, p.12).
- I distinguished deliberate debt from accidental mess, because the loan framing fits shortcuts taken knowingly for speed — accidental complexity is just a smell to refactor, with no strategic story attached (BeyondTechnicalDebt p.2, p.12).

Adapt to your lab

- Name [the debt you inherited in your change region: the long method, the duplication, the tangled dependency] and [how it slowed your change].
- Name [the shortcut you took, if any] and [the postfactoring commit that repaid it].

- If you ran [SonarLint/Sonar], cite [a number: issues, duplication %] as your snapshot — then note what the snapshot could not tell you (priority).
- State [your hotspot: the class you touched most often] and whether its complexity justified attention.

Key terms to drop

- **Technical debt** — substandard code in the way of current work, framed as a loan (BeyondTechnicalDebt p.2).
- **Principal / interest** — the substandard code (complexity, static) / the recurring cost (change frequency, behavioural) (BeyondTechnicalDebt p.12).
- **Hotspot** — complicated code you work with often; the first repayment target (BeyondTechnicalDebt p.12).
- **Lehman's laws** — continuing change; increasing complexity unless work counteracts it (BeyondTechnicalDebt p.3).
- **Snapshot vs movie** — a static violations report versus behavioural analysis over version-control history (BeyondTechnicalDebt p.6–10).
- **Complexity trend** — complexity rising over flat LOC = compounding debt (BeyondTechnicalDebt p.15).
- **Change coupling** — files that historically change together; temporal dependencies invisible to static analysis (BeyondTechnicalDebt p.19).
- **Knowledge loss / legacy** — code whose authors are gone; debt as an ownership problem (BeyondTechnicalDebt p.20–21).
- **Code decay vs reengineering** — the drift from Evolution into Servicing, and the refactoring counterforce that reverses it (Introduction p.21–22).

How did you verify the behaviour of your system (BDD and JGiven)?

Copy-paste answer

Beyond unit tests, I verified behaviour with Behaviour-Driven Development: specifying the system by its externally observable behaviour in a domain language readable by non-developers, executed like normal tests, producing living documentation (BDD p.4). Each requirement started as a user story — "As a [user type], I want [goal] so that [reason]" — which I mapped to a **Given/When/Then scenario**: Given the pre-conditions, When the action occurs, Then the expected outcome holds (BDDLab p.1; BDD p.5). I automated the scenarios with **JGiven**, the course's developer-friendly BDD framework: each scenario is a plain JUnit test method built from **stage classes** — typically one class each for Given, When, and Then — whose `snake_case` step methods render as prose in the generated reports (BDD p.9–11). State flows between stages through the scenario-state annotations: a Given stage provides a value with `@ProvidedScenarioState`, a later stage consumes it with `@ExpectedScenarioState` (BDD p.13). I chose JGiven over a classical plain-text framework like Cucumber deliberately: classical frameworks keep scenarios in separate `.feature` files glued to step definitions, which carries an additional maintenance cost, whereas JGiven scenarios live in Java with full IDE support — accepting the trade-off that domain experts can read but not author them (BDD p.6–7, p.20). Assertions in the Then steps used **AssertJ**, the fluent, actively maintained assertion library, so the checks read like sentences: `assertThat(result).contains(...)` (BDD p.22–25). The run output is the verification evidence: JGiven's console and HTML5 reports render each executed scenario back as readable Given/When/Then prose with pass/fail status, turning the test run itself into documentation that cannot go stale (BDD p.17–18). This is the change process's Verification phase made executable: the scenario is the acceptance criterion, and the change is done when its scenarios are green.

Why & how — justification add-ons

- I expressed acceptance criteria as Given/When/Then instead of prose requirements, because the three-part structure forces every test to state its assumptions, trigger, and expected outcome explicitly (BDD p.5).
- I chose JGiven over Cucumber because scenarios in the production language avoid the plain-text-plus-glue maintenance cost — one artifact instead of two kept in sync by regexes (BDD p.6–7).
- I built reusable stage classes instead of monolithic test methods, because a step written once is reused across scenarios, attacking test duplication the same way Extract Method attacks code duplication (BDD p.11).
- I used AssertJ rather than bare JUnit assertions because the fluent, type-specific API makes the Then step read as domain prose and produces self-explanatory failure messages (BDD p.22–25).
- I let the CI pipeline run the scenarios on every pull request, because acceptance criteria that only run on demand drift; wired into the build they are continuously enforced (BDD p.20; CILab p.1).
- I kept one scenario per user story instead of bundling behaviours, because a failing scenario should identify exactly one violated acceptance criterion — bundled scenarios fail ambiguously (BDDLab p.1; BDD p.5).

Adapt to your lab

- Quote [one of your real user stories] and [its Given/When/Then mapping] (BDDLab p.1).
- Name [your stage classes] and [one step method name] to show the prose-rendering effect.
- State [which state annotations you used between stages].
- If you tested the Swing GUI, mention [AssertJ-Swing: simulated interactions, reliable component lookup, screenshots of failures] (BDD p.29).
- Reference [the JGiven report output: scenarios total, all green] as your verification evidence.

Key terms to drop

- **BDD** — behaviour specified in shared domain language, defined collaboratively, executed as tests, producing living documentation (BDD p.4).
- **Given/When/Then** — pre-conditions, action, expected outcome; extra clauses with And (BDD p.5).
- **User story** — "As a [user type], I want [goal] so that [reason]" (BDDLab p.1).
- **JGiven** — developer-friendly Java BDD framework; scenarios are JUnit/TestNG methods (BDD p.9, p.20).
- **Stage class** — reusable per-phase step holder, unique to JGiven (BDD p.11).
- **@ProvidedScenarioState** / **@ExpectedScenarioState** — producer/consumer state transfer between stages (BDD p.13).
- **Living documentation** — the executing test doubles as always-current documentation (BDD p.4, p.17–18).
- **AssertJ** — fluent, polymorphic assertion library for the Then step (BDD p.22–25).
- **Classical vs developer-friendly frameworks** — Cucumber/JBehave (plain text, extra maintenance) versus Spock/JGiven (in-language) (BDD p.6–7).
- **Verification phase** — the change-process gate: acceptance criteria met, tests pass, no regressions (Software Testing p.81).

Why is software maintenance hard, and what did you learn from this course?

Copy-paste answer

Maintenance is hard for reasons that are *essential* — intrinsic to software, manageable but never removable (Introduction p.9–10). Software is **complex** (an enormous number of interacting parts), **invisible** (no natural geometry, so you reason through models and tools), **constantly changing** (it is the most malleable part of any system, so change pressure never stops), forced to **conform** to external interfaces it cannot redesign, and **discontinuous** (small edits can cause disproportionate behaviour jumps — why "trivial" changes regress) (Introduction p.9). On top of that, maintenance works on code you did not write and only partially understand: complete comprehension of a large system is impossible, so every change starts from deliberate partial understanding (ConceptLocation p.3). Unmanaged, these forces produce code decay — the slide from Evolution into Servicing, where only timid patches are affordable (Introduction p.21–22). What I learned from the course is that the answer to this hardness is *process and discipline*, not heroics. The phased change process gave me a repeatable route through an unfamiliar codebase: locate the concept, analyse impact, refactor before and after the change, propagate consistently, conclude into a verified baseline (ImpactAnalysis p.3). Refactoring and clean code keep the early phases cheap — clear names make location fast, low coupling keeps impact sets small (CleanCode p.11; OOPrinciples p.12). Automated tests and the CI pipeline make change *safe* by catching regressions within minutes (Software Testing p.81; ContinuousIntegration p.7). And the technical-debt lens made the economics explicit: structure is an investment whose return is the cheapness of the *next* change (BeyondTechnicalDebt p.12). The deepest single lesson, quantified by the Drawlets case: the cost of a change is the size of its propagation, not the lines edited — so everything in this course is ultimately about keeping propagation short (Drawlets p.37).

Why & how — justification add-ons

- I learned to refactor before changing rather than after things break, because prefactoring turns a hard change into an easy one — the cheapest time to restructure is before the new behaviour lands (Refactoring1 p.5).
- I learned to trust the process over intuition in unfamiliar code, because intuition fails at scale — the dependency-search and marking algorithms gave me termination guarantees my guesses never could (ConceptLocation p.10; ImpactAnalysis p.11–12).
- I learned that tests are a maintenance tool first and a quality tool second, because without the regression safety net neither refactoring nor actualization can be done with confidence (Software Testing p.81; BetterCode p.53–54).
- I learned to respect impact under-estimation as systemic, because even professionals missed two-thirds of actually-changed classes — humility about estimates is part of the method (Actualization p.27–28).
- I learned that maintainability is the point of design, because maintenance — mostly perfective change — dominates a system's lifetime cost, so code optimised only for shipping once is optimised for the wrong thing (SoftwareChange p.3; Introduction p.21–22).
- I learned that version control is a measurement instrument, not just a safety net, because the git history is what reveals hotspots, change coupling and knowledge loss — the behavioural data that prioritises all other maintenance work (BeyondTechnicalDebt p.9–12).

Adapt to your lab

- Replace the generic lessons with [your two most concrete personal lessons, each tied to a moment in your lab work].

- Name [the hardest moment of your project: a failed location, a surprise ripple, a broken build] and what it taught you.
- State [what you would do differently next time] — examiners read forward-looking reflection as understanding.
- Connect one lesson to [a specific artifact in your repository].
- Close with [the one-sentence version of your biggest lesson] — examiners remember endings.

Key terms to drop

- **Essential difficulties** — complexity, invisibility, changeability, conformity, discontinuity; manageable, never removable (Introduction p.9).
- **Accidental properties** — the replaceable technology-of-the-era concerns; improving them cannot remove the essentials (Introduction p.10).
- **Staged model** — initial development → evolution → servicing → phase-out → close-down (Introduction p.21).
- **Code decay / reengineering** — the drift into servicing, and the refactoring counter-force back to evolution (Introduction p.22).
- **Partial comprehension** — the as-needed strategy: minimum essential understanding (ConceptLocation p.3).
- **The change mini-cycle** — the phased process as the repeatable unit of maintenance (ImpactAnalysis p.3).
- **Propagation cost** — classes touched, not lines typed, is the cost of a change (Drawlets p.30, p.37).
- **Perfective maintenance** — the dominant kind: adding and improving functionality, not just fixing (SoftwareChange p.3).